

# RAG - Retrieval Augmented Generation

Kaja Wróblewska

8 czerwca 2025

## 1 Wprowadzenie

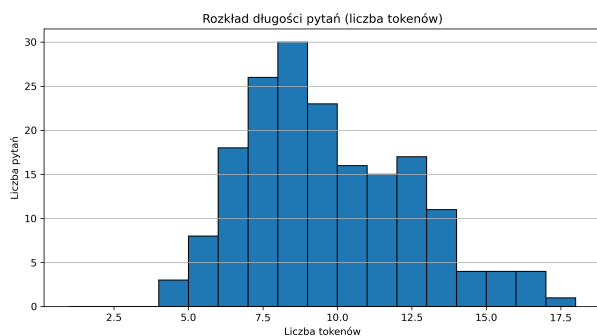
W poniższym sprawozdaniu przedstawiono porównanie modeli RAG (Retrieval Augmented Generation) odpowiadających na pytania na podstawie artykułów z Wikipedii.

## 2 Dane

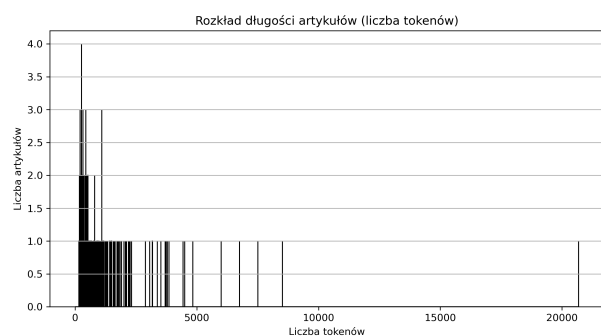
Przygotowano 300 artykułów z Wikipedia dumps. Teksty w języku polskim pobrano z *hugging-face*, wylosowano podzbiór i oczyszczono ze zbędnego formatowania. Artykuły zostały podzielone na 3 kategorie: krótkie (1000-2000 znaków), średnie (2000-4000 znaków) i długie: (>4000 znaków). Następnie przygotowano po 60 pytań do każdej kategorii. Kolejnym krokiem było przeprowadzenie następujących statystyk.

| Typ      | Ilość | Średnia długość | Mediana | Min | Maks |
|----------|-------|-----------------|---------|-----|------|
| Artykuły | 300   | 845.70          | 466.5   | 156 | 8513 |
| Pytania  | 180   | 9.21            | 9.0     | 4   | 17   |

Tabela 1: Statystyki ilości tokenów pytań i artykułów



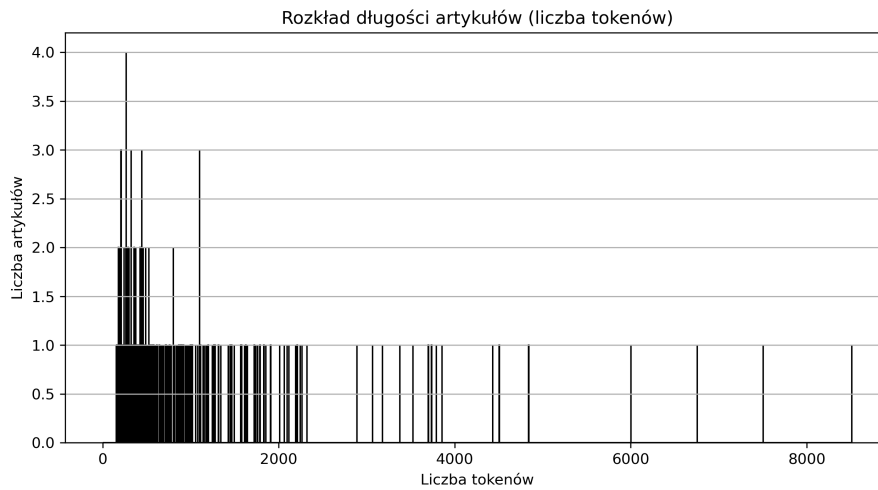
(a) Pytania



(b) Artykuły

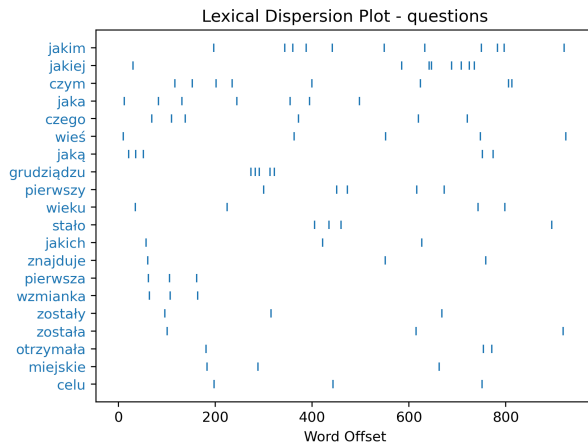
Rysunek 1: Rozkłady ilości tokenów

W trakcie analizy zaobserwowano wartość odstającą dotyczącą liczby tokenów w artykułach. W celu lepszego zbilansowania danych artykuł został wymieniony na krótszy. Po zamianie rozkład ilości tokenów wygląda następująco.

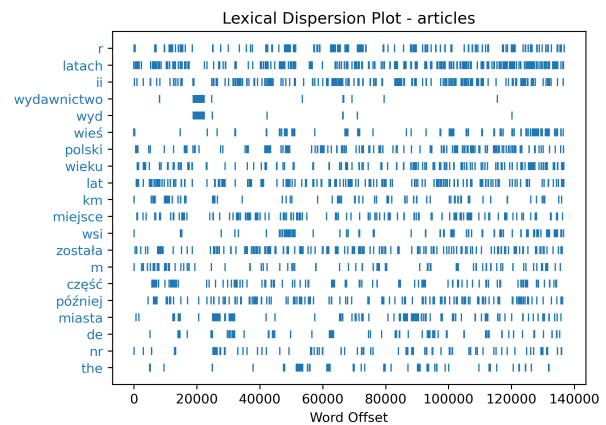


Rysunek 2: Rozkład ilości tokenów po usunięciu outliera

Następnym krokiem było zbadanie dyspersji (rozproszenia) 20 najczęstszych słów. Słowa puste (*stopwords*), liczby oraz znaki specjalne nie zostały uwzględnione. Najczęstszym słowem w artykułach było 'r'. Po analizie za pomocą konkordancji zidentyfikowano 'r' jako pozostałość po dacie (np. 1820 r.). Zostawiono token jako wyznacznik czasowy dla modelu *herBERT*.



(a) Pytania



(b) Artykuły

Rysunek 3: Wykresy rozproszenia

### 3 Model 1 - wektoryzatory

Pierwszy model opierał się o 3 wektoryzatory: Set of Words - zliczający unikalne wystąpienia słów, Bag of Words - zliczający ilość wystąpień oraz TF-IDF - uwzględniający zarówno częstość słowa w dokumencie, jak i jego unikalność w całym korpusie, by ocenić jego znaczenie. W modelu zastosowano następujące rozwiązania:

#### 1. Czyszczenie testu:

- Regex pozostawiający tylko litery i liczby
- Usunięcie stopwordsów

- Lematyzacja
- Entites nie zostały zastosowane - pogarszały wyniki

2. Normalizacja wektorów

3. Filtracja po artykułach: podobieństwo cosinusowe

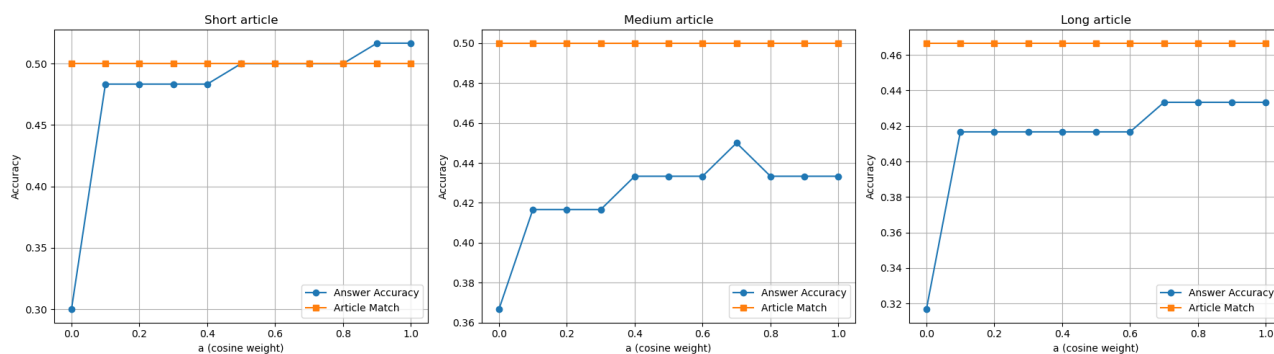
4. Filtracja po zdaniach artykułu: n-gramy, podobieństwo cosinusowe według wzoru:

$$a * \text{cos\_sim} + (1-a) * \text{ngram\_sim}$$

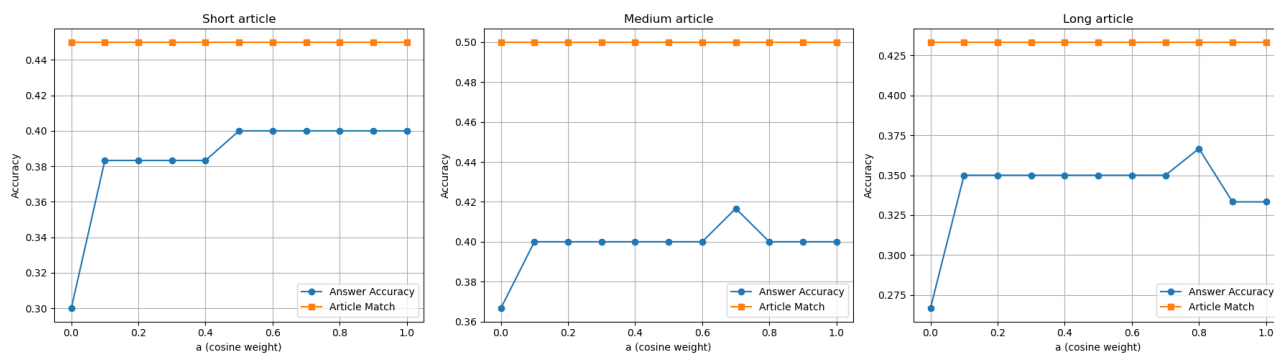
Następnie przeanalizowano wpływ parametru  $a$  na miary dokładności:

- Article match - ilość dobrze dopasowanych artykułów do pytania / ilość wszystkich prób dopasowania
- Answer Accuracy - poprawne odpowiedzi / wszystkie odpowiedzi

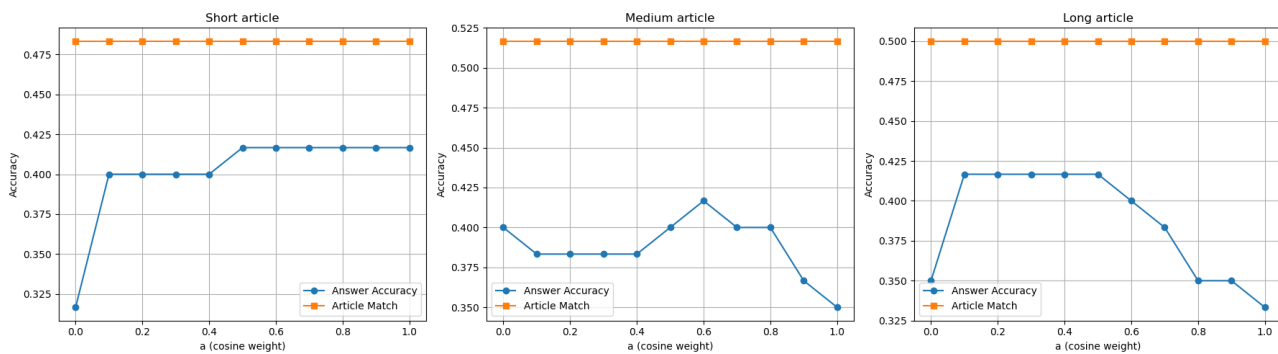
Poprawna odpowiedź była klasyfikowana jako podobieństwo cosinusowe  $\geq 0.9$  między predykcją a kluczem.



Rysunek 4: Set of Words - wyniki



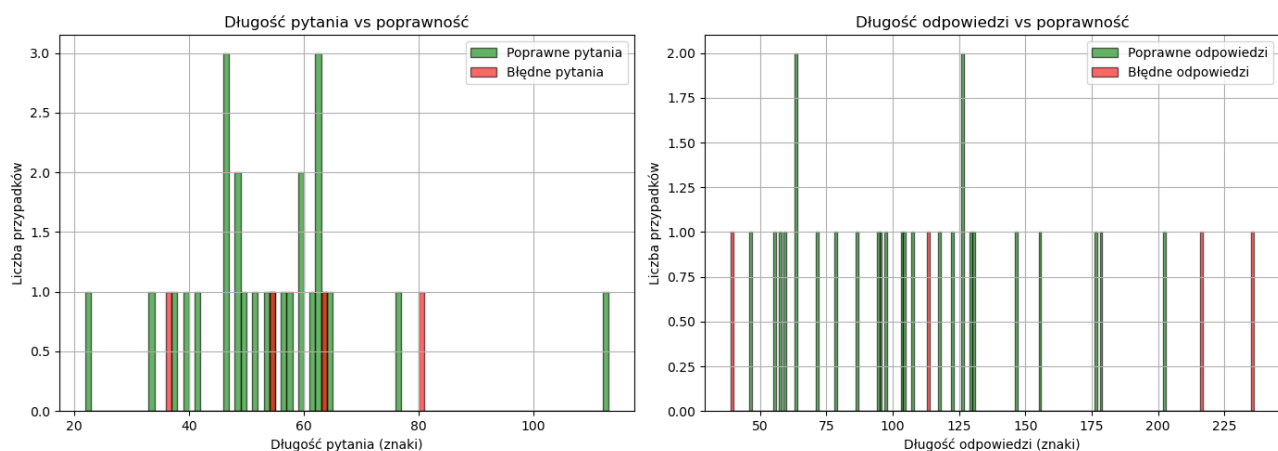
Rysunek 5: Bag of Words - wyniki



Rysunek 6: TF-IDF - wyniki

Nie zauważono globalnego trendu dla parametru  $a$ , niemniej jednak Model 1 lepiej reagował na wyższe wartości  $a \geq 0.7$ . Co więcej, najprostszy wektoryzator *[Set of Words]* osiągnął najlepsze wyniki. Możliwym powodem jest odporność wektoryzatora na powtórzenia i rzadkie słowa.

Dodatkowo przeanalizowano wpływ długości pytań i odpowiedzi na dokładność predykcji modelu. Wykorzystano tylko zbiór pytań z poprawnie przypisanym artykułem. Nie zaobserwowano wpływu długości pytania na dokładność, natomiast istnieje sugestia, że dłuższe odpowiedzi były trudniejsze do przewidzenia. Jednakże ze względu na małą ilość danych, ciężko jest wysunąć jednoznaczne wnioski. Poniżej znajduje się przykładowy wykres z wektoryzatora Set of Words o najlepszych wynikach.



Rysunek 7: Wpływ długości pytań i odpowiedzi na dokładność

### 3.1 Potencjalne modyfikacje

Najlepszą metodą było by przeprowadzanie porównania parametrów z *GridSearchCV*, niestety jest to bardzo czasochłonna technika. Poniżej przedstawiam zestaw parametrów wartych do przetestowania.

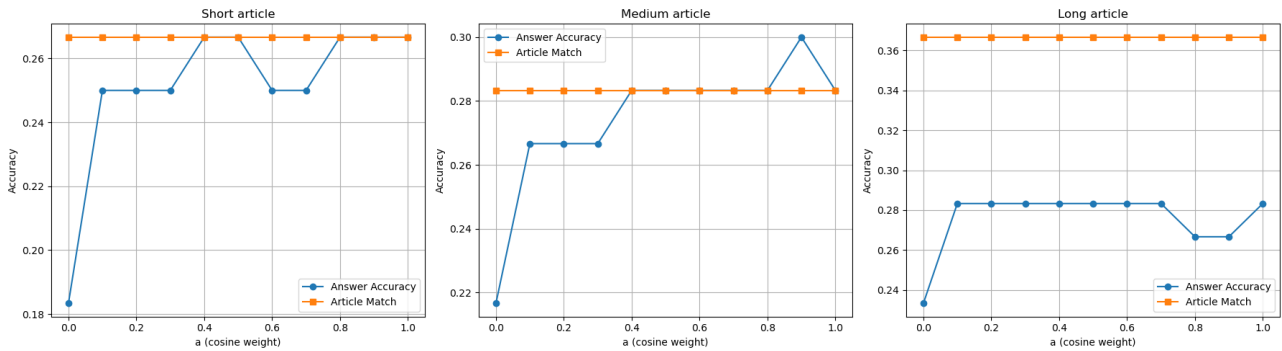
#### 1. Wszystkie wektoryzatory

- $min\_df$ : ignorowanie bardzo rzadkich słów
- $max\_df$ : ignorowanie bardzo częstych słów (szumu)
- $max\_features=5000$ : ograniczenie ilości cech

## 2. TF-IDF

- *sublinear\_tf=True*: zamiana liczenia tf (term frequency) z liniowego na logarytmiczne
- *binary=True*: zmuszenie wektoryzatora do używania binarnych wartości
- sprawdzenie efektywności *normalize(...)* z *sklearn.preprocessing*

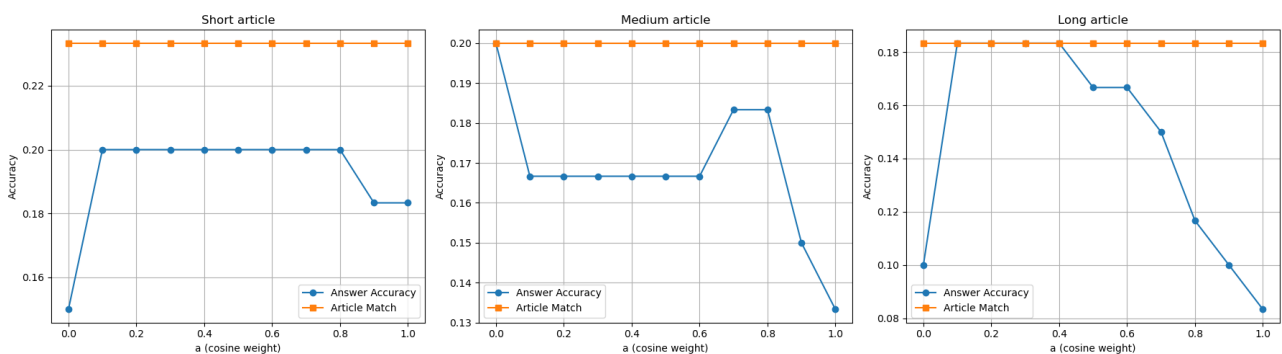
Przeprowadzono przykładową symulację z dodanymi parametrami do wersji z wektoryzatorem TF-IDF. Niestety, arbitralne dobranie parametrów obniżyło wyniki. Wybrane parametry to: *min\_df=3*, *max\_df=0.9*, *sublinear\_tf=True* i *max\_features=5000*



Rysunek 8: TF-IDF z dodatkowymi parametrami

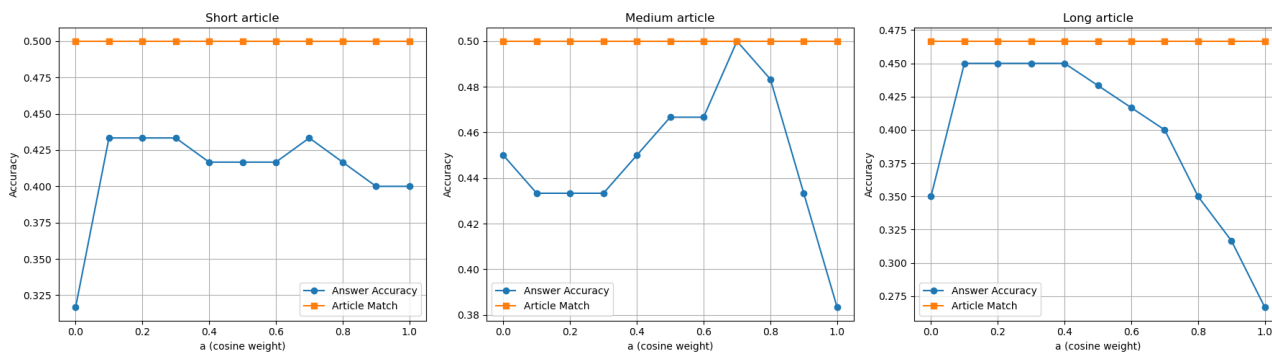
## 4 Model 2 - osadzenia *spaCy*

Kolejna iteracja modelu dotyczyła zamiany wektoryzatora na gotowe osadzenia *spaCy*. Jednakże okazało się, że wyniki są znacznie gorsze. Potencjalnym powodem może być charakter osadzeń - ogólne i „rozmyte”, skupiają się na sensie, a nie na dokładnych dopasowaniach słów. Mimo że *spaCy* był trenowany na m. in. korpusie Wikipedii, RAG wymaga precyzyjnego dopasowania krótkich fragmentów.



Rysunek 9: *spaCy* embeddings - wyniki

Próba zaadresowania braków osadzeń *spaCy*, było użycie wektoryzatora do filtracji artykułów. Przy filtracji zdań z dużo krótszego tekstu, osadzenia radziły sobie znacznie lepiej. Udało się nawet osiągnąć wynik porównywalny do wektoryzatorów: 0.5. Jednakże, dokładność była dużo mniej stabilna. Co ciekawe, Model 2 wykazał odwrotny trend parametru  $a$ .



Rysunek 10: *spaCy* embeddings with Set of Words - wyniki

## 4.1 Potencjalne modyfikacje

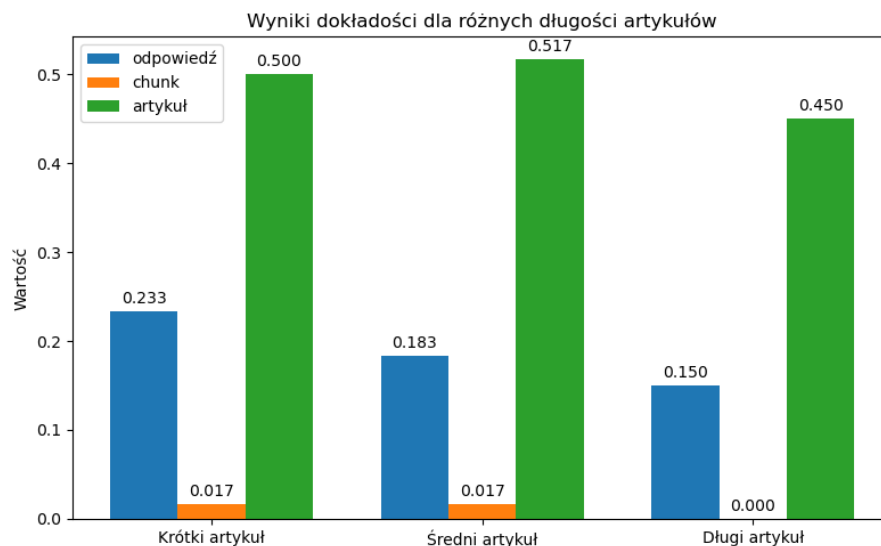
Warto spróbować wyszukiwania najlepszych kawałków tekstu (*chunks*) (np. 300 znaków) zamiast najlepszych artykułów i wyszukiwania pasującej odpowiedzi z  $k$  najlepszych kawałków. Rozwiązanie pasuje do obu wcześniejszych modeli i zostało przetestowane na modelu *SBERT* ze względu na ograniczenie maksymalnej długości wejścia.

## 5 Model 3 - *SBERT*

Ostatnią klasą modeli były modele *SBERT*, a dokładnie *sentence-transformers/paraphrase-multilingual-mpnet-base-v2*. Jest to wielojęzyczny model sieci neuronowej wywodzący się z architektury *BERT*, wykorzystujący ulepszoną wersję – *MPNet*. Model ten jest częścią biblioteki *sentence-transformers*, która specjalizuje się w tworzeniu reprezentacji semantycznych całych zdań (*sentence embeddings*). Został wytrenowany do znajdowania semantycznie podobnych zdań i zwraca wektory, które można porównywać za pomocą miar podobieństwa, np. podobieństwa cosinusowego. Głównym ograniczeniem modelu jest maksymalna długość wejścia – 512 tokenów. Model składał się z poszczególnych etapów:

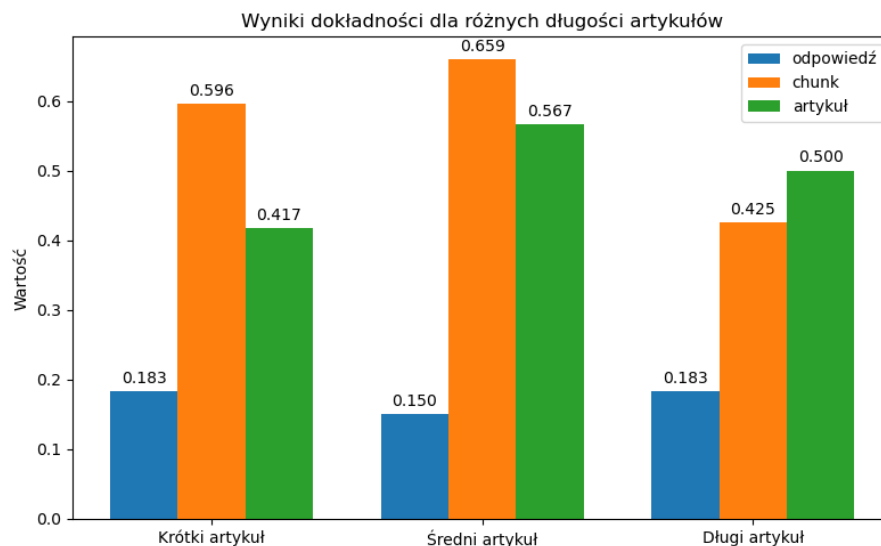
1. Preprocessing - sprawdzono wyniki z (lematyzacja + no stopwords) i bez
2. Chunki - dzielenie artykułów na nakładające się chunki
3. Wybranie zdania z najlepszego chunku

Wyniki prezentowały się następująco:



Rysunek 11: Wyniki dokładności BERT 1

W kolejnej iteracji modelu wybierano odpowiedź z *top-k* najlepszych chunków. Zwiększyło to występowanie dobrej odpowiedzi w chunkach, ale nie polepszyło predykcji odpowiedzi.



Rysunek 12: Wyniki dokładności BERT 2

## 5.1 Potencjalne modyfikacje

Warto przetestować różne wartości *chunk\_size* i *stride* przy użyciu *GridSearchCV*, a także połączenie sieci neuronowych z prostszymi modelami.

## 6 Podsumowanie

W powyższym sprawozdaniu przeprowadzono analizę trzech podejść do implementacji modeli RAG na podstawie polskich artykułów z Wikipedii.

Pierwszy model wykorzystał tradycyjne wektoryzatory (Set of Words, Bag of Words, TF-IDF) i wykazał najlepszą dokładność, zwłaszcza przy większych wartościach parametru  $a$ .

Drugi model bazował na osadzeniach *spaCy* i osiągał gorsze wyniki, co jest najprawdopodobniej wynikiem bardziej ogólnego charakteru wektorów, które są mniej precyzyjne w krótkich fragmentach tekstu. Użycie osadzeń *spaCy* do filtracji artykułów i klasycznych wektoryzatorów do filtracji zdań przyniosło umiarkowaną poprawę, jednakże dokładność była mniej stabilna.

Trzeci model oparty na sieciach neuronowych z użyciem *SBERT*. Ze względu na ograniczenia modelu, chunkowanie było koniecznym krokiem do osiągnięcia wyników wyższych od zera. Zwiększenie rozmiaru i liczby rozpatrywanych chunków poprawiło wyznaczenie prawidłowego fragmentu tekstu, ale przełożyło się jednoznacznie na wzrost ogólnej dokładności modelu.

W projekcie przetestowano wiele iteracji modeli używając różnych technologii tworzenia modelu RAG. Biorąc pod uwagę, że losowe prawdopodobieństwo wybrania poprawnego artykułu to 0.1, a wybranie poprawnej odpowiedzi to kolejno 0.0006, 0.0003, 0.0001 dla każdej kategorii, wyniki są całkiem zadowalające.

W kolejnych iteracjach warto rozszerzyć zbiór danych na wszystkie artykuły oraz przetestować parametry przy użyciu *GridSearchCV*.